

Unidad 03 – Variables, convenciones y operadores

Cátedra: Programación en Computación – UTN FRRQ Docentes: Longhi Pablo, Talijancic Iván

De qué se trata esta unidad

En la unidad 02 usamos variables sin explicarlas: `const tension = 220` y listo. Hoy vemos **por qué** se escriben así y qué se puede hacer con ellas.

Al terminar vas a poder:

- 🗂 Elegir entre `let` y `const` con criterio, y saber por qué **no** usamos `var`
- 📌 Nombrar variables como se espera en el curso y en los exámenes
- 📊 Distinguir los **tipos de datos** y averiguar el tipo de cualquier valor
- ➕ Usar todos los **operadores**: aritméticos, de asignación, relacionales y lógicos
- ✅ Escribir una **verificación** que responda `true` o `false`
- 🧠 Entender por qué `'20' + 5` da `'205'` y `'20' - 5` da `15`

🔑 **Lo que se prepara hoy:** los operadores relacionales y lógicos son la materia prima de los condicionales de la unidad 04. Hoy sólo los imprimimos; la clase que viene deciden qué hace el programa.

1. 🗂 Declarar variables: `let`, `const` y `var`

Una variable es un nombre para un dato. Hay tres formas de declararla, y **usamos dos**.

1.1 `const` – el valor no se reasigna

```
const tension = 220

console.log(tension)
```

```
220
```

Si intentás reasignarla, el programa **falla**:

```
const tension = 220
tension = 380
```

```
TypeError: Assignment to constant variable.
```

1.2 `let` — el valor puede cambiar

```
let contador = 0
contador = 1
contador = 2

console.log(contador)
```

```
2
```

1.3 La regla de la cátedra

🔑 `const` por defecto. `let` sólo cuando el valor tiene que cambiar.

No es un capricho de estilo: cuando lees `const`, ya sabés que ese valor **no se toca en todo el programa**. Es una garantía que te da el lenguaje y que te ahorra rastrear el código para ver si alguien lo modificó.

En la práctica, casi todo es `const`. `let` aparece en acumuladores y contadores — que son justamente lo que vamos a ver en la unidad 05 con los bucles.

1.4 ⚠️ `var`: por qué no lo usamos

`var` es la forma vieja, de antes de 2015. Sigue funcionando, pero tiene dos comportamientos que generan errores difíciles de encontrar.

Problema 1: se puede usar antes de declararla.

```
console.log(nombre)
var nombre = 'Ana'
```

```
undefined
```

No falla: imprime `undefined`. Con `let` o `const`, el mismo código **te avisa**:

```
ReferenceError: Cannot access 'nombre' before initialization
```

Y un error que avisa es mucho mejor que un `undefined` silencioso que se propaga.

Problema 2: ignora los bloques.

Un **bloque** es todo lo que está entre llaves `{ }`. Con `let` y `const`, la variable existe sólo dentro del bloque donde se declaró:

```
{
  const mensaje = 'adentro del bloque'
  console.log(mensaje)
}
```

adentro del bloque

Si intentás usarla afuera, falla — y eso es **bueno**: te avisa que estás usando algo que no corresponde. `var`, en cambio, se escapa del bloque y queda disponible en toda la función, lo que produce colisiones de nombres.

✂ **En resumen:** `var` no está prohibido por el lenguaje, está prohibido **en esta cátedra**. Vas a verlo en código viejo y en tutoriales de internet; ahora sabés por qué no lo copiás.

	CONST	LET	VAR
¿Se puede reasignar?	✗	✓	✓
¿Respetar los bloques <code>{ }</code> ?	✓	✓	✗
¿Avisa si la usás antes de declararla?	✓	✓	✗
¿La usamos?	Sí, por defecto	Sí, si cambia	No

2. 📝 Convenciones de nombres

El lenguaje te deja nombrar casi como quieras. La **convención** es lo que hace que tu código se parezca al de todos los demás — y a lo que se espera en un parcial.

2.1 `camelCase` — variables y funciones

Primera palabra en minúscula, cada palabra siguiente arranca con mayúscula. **Sin espacios, sin guiones.**

```
const nominalVoltage = 380
const dailyHours = 8
const isWithinRange = true
```

2.2 `PascalCase` — clases

Igual que `camelCase`, pero la primera palabra **también** con mayúscula.

```
class ElectricMotor {  
  // ...  
}
```

🔑 No vamos a escribir clases en este curso. La convención se menciona para que reconozcas qué es cada cosa cuando leas código.

2.3 UPPER_SNAKE_CASE – constantes del dominio

Todo en mayúsculas, separado con guiones bajos. Se reserva para valores **fijos del problema**: límites, tolerancias, constantes físicas.

```
const NOMINAL_VOLTAGE = 380  
const TOLERANCE = 0.05  
const MAX_TEMPERATURE = 85
```

2.4 🇬🇧 Los identificadores van en inglés

Regla de la cátedra: **los nombres de variables y funciones se escriben en inglés**; los textos que ve el usuario, en español.

```
// ✅ así  
const dailyHours = parseFloat(prompt('Horas de uso por día: '))  
  
// ❌ así no  
const horasDiarias = parseFloat(prompt('Horas de uso por día: '))
```

Es lo que se usa en la industria y lo que se espera en los exámenes.

2.5 Nombres descriptivos

```
// ❌ ilegible  
const x = 380  
const y = 4.2  
console.log(x * y)  
  
// ✅ se explica solo  
const voltage = 380  
const current = 4.2  
console.log(voltage * current)
```

🔑 **La prueba:** si alguien lee tu línea sin ver el resto del programa, ¿entiende qué es ese valor? Si no, el nombre está mal.

3. ¹²₃₄ Tipos de datos

Todo valor en JavaScript tiene un **tipo**. Los que nos importan son cinco.

TIPO	QUÉ GUARDA	EJEMPLO
<code>number</code>	Números, con o sin decimales	<code>220</code> , <code>4.2</code> , <code>-15</code>
<code>string</code>	Texto	<code>'M-14'</code> , <code>'Ana'</code>
<code>boolean</code>	Verdadero o falso	<code>true</code> , <code>false</code>
<code>undefined</code>	Declarada, pero sin valor asignado	<code>undefined</code>
<code>null</code>	Vacío a propósito	<code>null</code>

3.1 `typeof` – averiguar el tipo

```
const voltage = 380
const motorId = 'M-14'
const isRunning = true
let notAssigned

console.log(typeof voltage)
console.log(typeof motorId)
console.log(typeof isRunning)
console.log(typeof notAssigned)
```

```
number
string
boolean
undefined
```

💡 **`typeof` es tu herramienta de diagnóstico.** Cuando una cuenta da un resultado imposible, lo primero que hacés es imprimir el `typeof` de lo que estás sumando. Nueve de cada diez veces, algo que creías número es texto.

3.2 Un solo tipo para todos los números

`220` y `4.2` son los dos `number`. **No hay un tipo entero y otro decimal**, como en otros lenguajes. Por eso `parseInt()` y `parseFloat()` no devuelven tipos distintos: las dos devuelven `number`, sólo cambia si conservan los decimales.

3.3 `undefined` vs `null`

- `undefined` — **nadie le asignó nada**. Es lo que pasa cuando declarás y no inicializás.
- `null` — **está vacío a propósito**. Sos vos el que decidió poner "acá no hay dato".

```
let temperature      // undefined: todavía no medí
const noSensor = null // null: decidí que este canal no tiene sensor

console.log(temperature)
console.log(noSensor)
```

```
undefined
null
```

😬 **Una rareza que vas a ver:** `typeof null` devuelve `'object'`, no `'null'`. Es un error del diseño original de JavaScript, de 1995, que nunca se corrigió para no romper la web entera. No hay nada que entender: es así, y conviene saberlo antes de que te desconcierte.

4. ÷ Operadores aritméticos

OPERADOR	QUÉ HACE	EJEMPLO	RESULTADO
+	Suma	<code>10 + 3</code>	<code>13</code>
-	Resta	<code>10 - 3</code>	<code>7</code>
*	Multiplicación	<code>10 * 3</code>	<code>30</code>
/	División	<code>10 / 3</code>	<code>3.333...</code>
%	Resto de la división	<code>10 % 3</code>	<code>1</code>
**	Potencia	<code>10 ** 3</code>	<code>1000</code>

4.1 ★ El resto (%) es más útil de lo que parece

% devuelve **lo que sobra** de una división entera. `10 % 3` es `1` porque 3 entra tres veces en 10 y sobra 1.

Suena académico, pero resuelve dos problemas que vas a tener todo el curso:

Saber si un número es par: un número es par si al dividirlo por 2 no sobra nada.

```
const measurements = 30

console.log(measurements % 2)
console.log(measurements % 2 === 0)
```

```
0
true
```

Repartir en grupos: si tenés 17 sensores y los agrupás de 5 en 5, `17 % 5` te dice cuántos quedan sueltos.

```
console.log(17 % 5)
```

```
2
```

4.2 Precedencia: primero `*` `/` `%`, después `+` `-`

Como en matemática. Y como en matemática, **los paréntesis mandan**.

```
console.log(2 + 3 * 4)
console.log((2 + 3) * 4)
```

```
14
20
```

💡 **Consejo práctico:** ante la mínima duda, poné paréntesis. No cuestan nada y hacen que la intención quede escrita. Nadie va a criticarte por un paréntesis de más; sí por un resultado mal calculado.

5. 🖋️ Operadores de asignación

= asigna. No es "igual" en el sentido matemático: es "guardá esto acá".

```
let counter = 0
counter = counter + 1

console.log(counter)
```

```
1
```

Esa línea `counter = counter + 1` se lee de derecha a izquierda: *tomá lo que hay en `counter`, sumale 1, y guardalo de nuevo en `counter`*.

Como es muy frecuente, hay atajos:

ATAJO

EQUIVALE A

`counter += 1``counter = counter + 1``counter -= 1``counter = counter - 1``total += price``total = total + price``counter++``counter = counter + 1``counter--``counter = counter - 1`

```
let total = 0
total += 100
total += 50

console.log(total)
```

150

⚠ Todos estos **modifican** la variable, así que sólo funcionan con `let`. Con `const` dan `TypeError`.

6. ⚖ Operadores relacionales

Comparan dos valores y devuelven **siempre** un `boolean`: `true` o `false`.

OPERADOR

PREGUNTA

`>`

¿es mayor que?

`<`

¿es menor que?

`>=`

¿es mayor o igual?

`<=`

¿es menor o igual?

`===`

¿son iguales?

`!==`

¿son distintos?

```
const measured = 219.4
const nominal = 220

console.log(measured < nominal)
console.log(measured >= 200)
console.log(measured === nominal)
console.log(measured !== nominal)
```



```
true
true
false
true
```

🔑 **Un operador relacional no decide nada.** Sólo responde una pregunta con `true` o `false`. Quien decide qué hacer con esa respuesta es el `if` de la unidad 04.

6.1 ⚠️ `===` y no `==`

JavaScript tiene dos operadores de igualdad, y **usamos uno solo**.

```
console.log(5 === 5)
console.log('5' === 5)
console.log('5' == 5)
```

```
true
false
true
```

Mirá la última línea: `==` dice que el **texto** `'5'` es igual al **número** `5`. Antes de comparar, convierte los tipos por su cuenta.

Eso parece cómodo y es una fuente inagotable de errores: el día que compares algo que venía de un `prompt()` (que es texto) con un número, `==` te va a decir que son iguales aunque uno sea `'5'` y el otro `5` — y el bug va a aparecer tres líneas más abajo, cuando intentes sumarlos.

🔑 **Regla de la cátedra: siempre `===` y `!==`.** Comparan valor y tipo, sin conversiones a tus espaldas. Se corrige como error grave.

7. 🔗 Operadores lógicos

Combinan condiciones. También devuelven `boolean`.

OPERADOR	NOMBRE	DA <code>TRUE</code> CUANDO...
<code>&&</code>	Y	las dos son verdaderas
<code> </code>	O	al menos una es verdadera
<code>!</code>	No	invierte: <code>!true</code> es <code>false</code>

7.1 Un caso de ingeniería

La norma admite una tensión de línea de 380 V con $\pm 5\%$ de tolerancia. ¿La medición está en rango?

```
const NOMINAL_VOLTAGE = 380
const TOLERANCE = 0.05
const measured = 372.5

const lowerLimit = NOMINAL_VOLTAGE * (1 - TOLERANCE)
const upperLimit = NOMINAL_VOLTAGE * (1 + TOLERANCE)

console.log(`Límites: ${lowerLimit} V a ${upperLimit} V`)
console.log(measured >= lowerLimit && measured <= upperLimit)
```

```
Límites: 361 V a 399 V
true
```

Las dos condiciones tienen que cumplirse a la vez, así que va `&&`.

7.2 `||` – alcanza con una

Un motor se marca para revisión si lleva más de 10.000 horas o si la temperatura pasó los 85 °C. Cualquiera de las dos alcanza:

```
const hours = 4200
const temperature = 91

console.log(hours > 10000 || temperature > 85)
```

```
true
```

7.3 `!` – negar

```
const isRunning = false

console.log(!isRunning)
```

```
true
```

💡 Se lee "no está corriendo". Leer los `!` en voz de pregunta ayuda: `!isRunning` es "¿no está corriendo?".

7.4 Precedencia: `!` primero, después `&&`, después `||`

```
const a = true
const b = false
const c = true

console.log(a || b && c)
console.log((a || b) && c)
```

```
true
true
```

Dan lo mismo por casualidad. Con otros valores no. 🙅 **Poné paréntesis.**

8. 🤖 Coerción de tipos

Ya te pasó en la unidad 02: `'20' + 5` da `'205'`. Ahora vemos por qué, porque el comportamiento **no es el mismo con todos los operadores**.

```
console.log('20' + 5)
console.log('20' - 5)
console.log('20' * 5)
console.log('20' / 5)
```

```
205
15
100
4
```

El `+` **es el raro**. Sirve para dos cosas distintas: sumar números y **pegar** textos. Si uno de los dos lados es texto, gana el pegado.

Los demás (`-`, `*`, `/`, `%`) no tienen ese doble uso, así que convierten el texto a número y hacen la cuenta.

🔑 **Esto explica la regla de la unidad 02.** No es que `prompt()` esté roto: es que devuelve `string`, y con `+` el texto gana. Por eso **siempre** `parseInt()` o `parseFloat()` cuando el dato es un número.

```
console.log(typeof '20')
console.log(typeof parseFloat('20'))
```

```
string
number
```

8.1 NaN – "esto no es un número"

Si le pedís a `parseFloat()` que convierta algo que no es un número, no falla: devuelve `NaN` (*Not a Number*).

```
const value = parseFloat('cuatro')

console.log(value)
console.log(typeof value)
```

```
NaN
number
```

Dos cosas raras y ciertas: `NaN` es de tipo `number`, y **cualquier cuenta que lo toque da `NaN`**. Si un resultado te sale `NaN`, buscá el `parseFloat` que recibió texto que no era un número.

9. 🐛 Errores comunes

ERROR	SÍNTOMA	SOLUCIÓN
Reasignar una <code>const</code>	<code>TypeError: Assignment to constant variable</code>	Si el valor cambia, usá <code>let</code>
Usar <code>==</code> en lugar de <code>===</code>	Compara <code>'5'</code> con <code>5</code> y dice que son iguales	Siempre <code>===</code>
Usar <code>=</code> donde va <code>===</code>	Asigna en vez de comparar	<code>=</code> asigna · <code>===</code> compara
Olvidar <code>parseFloat</code>	<code>'20' + 5</code> da <code>'205'</code>	Convertí antes de calcular
<code>NaN</code> en el resultado	Toda la cuenta da <code>NaN</code>	Buscá el <code>parseFloat</code> que recibió texto
Nombres de una letra	Nadie entiende el código	<code>voltage</code> , no <code>v</code>
Confiar en la precedencia	Resultado inesperado	Paréntesis
Declarar con <code>var</code>	Se corrige como error	<code>const</code> , o <code>let</code> si cambia

10. Resumen

CONCEPTO	PARA QUÉ
<code>const</code>	Por defecto. El valor no se reasigna
<code>let</code>	Sólo si el valor tiene que cambiar
<code>var</code>	No se usa en la cátedra
<code>camelCase</code>	Variables y funciones
<code>UPPER_SNAKE_CASE</code>	Constantes del problema
<code>typeof</code>	Averiguar el tipo de un valor. Herramienta de diagnóstico
<code>+</code> <code>-</code> <code>*</code> <code>/</code>	Aritmética
<code>%</code>	Resto: paridad y agrupamientos
<code>+=</code> <code>++</code>	Atajos de asignación. Sólo con <code>let</code>
<code>></code> <code><</code> <code>>=</code> <code><=</code>	Comparar. Devuelven <code>boolean</code>
<code>===</code> <code>!==</code>	Igualdad estricta . Nunca <code>==</code>
<code>&&</code> <code> </code> <code>!</code>	Combinar condiciones
<code>NaN</code>	Una conversión que falló

11. Para la próxima clase

En la **unidad 04** llegan los **condicionales**: `if`, `else if`, `else` y `switch-case`. Ahí todo lo de hoy se pone en uso — las comparaciones y los `&&` dejan de imprimirse y empiezan a **decidir** qué hace el programa.

Para llegar preparado:

1.  Hacé el TP de esta unidad, sobre todo los problemas con `&&` y `||`
2.  Asegurate de tener claro `===` **contra** `==`: es lo que más se cobra en el parcial
3.  Repasá el `%`: en la unidad 05 lo vas a usar en cada bucle

Material de la unidad

- **Presentación de clase** (<https://github.com/italijancic/pc-2026/blob/main/unidades/03-variables-y-operadores/presentacion.pdf>)

- **Trabajo Práctico** (<https://github.com/italijancic/pc-2026/blob/main/unidades/03-variables-y-operadores/tp.pdf>)
- **Ejemplos de la clase** (<https://github.com/italijancic/pc-2026/tree/main/unidades/03-variables-y-operadores/ejemplos>)